

Discrete Logarithm Distributed Key Generation

NEAR One

Abstract. TODO

Table of Contents

1	Introduction	3
2	Preliminaries	3
	2.1 Polynomial Interpolation and Shamir Secret Sharing	3
	2.2 Communication Model	4
	2.3 Distributed Key Generation	5
	2.4 Online-Extractable & Straight-Line Simulation Proof System ...	5
3	PedPoP	6
4	PedPoP+	6
	4.1 Key Generation	7
	4.2 Key Resharing	7
5	Security	8

1 Introduction

In this work, we define a variant of PedPoP, a two-round DKG introduced by Komlo and Goldberg [EPRINT:KomGol20]. Our variant, PedPoP+, achieves the notion of simulatability with aborts, a stronger notion of security than plain PedPop.

2 Preliminaries

Notation. We use $\lambda \in \mathbb{Z}$ to denote the security parameter. We use $x \leftarrow y$ to denote the assignment of the value of y to x , and write $x \leftarrow_s S$ to denote sampling an element from the set S independently and uniformly at random. For a randomized algorithm $\mathcal{A}$ we write $y \leftarrow_s \mathcal{A}(x)$ to denote the random variable that is the output of $\mathcal{A}(x)$ on input x . We use $[n]$ for the set $\{1, \dots, n\}$. For a positive integer q we use $\mathbb{Z}_p$ to denote the ring $\mathbb{Z}/q\mathbb{Z}$.

When writing protocols, **require** denotes the clause that if some predicate $\cdot$ does not hold, then the respective party must abort the protocol.

2.1 Polynomial Interpolation and Shamir Secret Sharing

A polynomial $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$ of degree $t-1$ over a field $\mathbb{F}$ can be interpolated by t points. Let $\eta \subseteq [n]$ be the list of t distinct indices corresponding to the x -coordinates $x_i \in \mathbb{F}$ of these points. Then the Lagrange polynomial $L_i(x)$ has the form:

$$L_i(x) = \prod_{j \in \eta; j \neq i} \frac{x - x_j}{x_i - x_j}$$

Given a set of t points $(x_i, f(x_i))_{i \in \eta}$, any point $f(\tilde{x})$ on the polynomial f can be determined by Lagrange interpolation as follows:

$$f(\tilde{x}) = \sum_{k \in \eta} f(x_k) \cdot L_k(\tilde{x})$$

Shamir Secret Sharing. Shamir secret sharing [Shamir79] is an (n, t) -threshold secret sharing scheme consisting of algorithms (Share, Recover), defined as follows:

- **Share** $(x, n, t) \rightarrow \{(1, \bar{x}_1), \dots, (n, \bar{x}_n)\}$: On input a secret x , number of participants n , and threshold t , perform the following. First, define a polynomial $f(Z) = x + a_1Z + a_2Z^2 + \dots + a_{t-1}Z^{t-1}$ with $a_1, \dots, a_{t-1} \leftarrow_s \mathbb{Z}_p$. Then, set each participant's share $\bar{x}_i, i \in [n]$, to be the evaluation of $f(i)$:

$$\bar{x}_i \leftarrow x + \sum_{j \in [t-1]} a_j i^j$$

Output $\{(i, \bar{x}_i)\}_{i \in [n]}$.

- **Recover**($t, \{(i, \bar{x}_i)\}_{i \in C}$) $\rightarrow \perp/x$: On input threshold t and a set of shares $\{(i, \bar{x})\}_{i \in C}$, output $\perp$ if the coalition $C \not\subseteq [n]$ or if $|C| < t$. Otherwise, recover x as follows:

$$x \leftarrow \sum_{i \in C} \bar{x}_i \cdot L_i(0)$$

where the Lagrange coefficient for the set C is defined by

$$L_i(0) = \prod_{j \in C, j \neq i} \frac{j}{j - i}$$

2.2 Communication Model

When describing distributed key generation schemes, we assume each pair of parties can communicate over a secure channel. We introduce below two types of secure channels that we rely upon for our DKG construction.

We also assume a synchronous model of communication; which means we require that protocol execution progresses in well-defined rounds, with every message sent in a given round being delivered before the start of the subsequent round. We assume that the adversary can be rushing, meaning that in any given Round r the corrupted parties can receive all messages sent by the honest parties in Round r before the corrupted parties are required to send their Round- r messages.

Broadcast Channels A broadcast channel sends one message to all other players, with the guarantee that all players received the same message. A broadcast channel guarantees the properties of authenticity, reliability, and consistency. We denote sending over a broadcast channel with **sendbroadcast**. Note that on the MPC level, these channels are equipped with timeouts.

Concretely, we instantiate **sendbroadcast** with the “Authenticated Double-Echo Broadcast” protocol from [CGR11] in order to achieve Byzantine reliable broadcast. A reliable broadcast channel specifically guarantees the properties of Validity, No duplication, Integrity and totality (see Module 3.11 and Module 3.12 of [CGR11]). The file *echo_broadcast.rs* contains the implementation of the reliable broadcast protocol. We essentially follow the same algorithm 3.18 of [CGR11] with three mild differences:

1. If a party is broadcasting a value, then the protocol expects that other parties would also want to broadcast some values. This is essentially equivalent to running n parallel instances of algorithm 3.18 where one instance where party k is a sender and $n-1$ instances where k is a receiver. Such an implementation serves directly our use cases in protocols where symmetry holds between participants. Additionally, this implementation prevents having deadlocks (two parties waiting for each others to send values on two different broadcast channels) and prevents having inefficient sequential broadcast instances.
2. If an (honest) broadcast sender recovers an output at the end of the protocol that is different than the value it input, then it aborts considering that the effective number of malicious parties is larger than what the protocol allows.

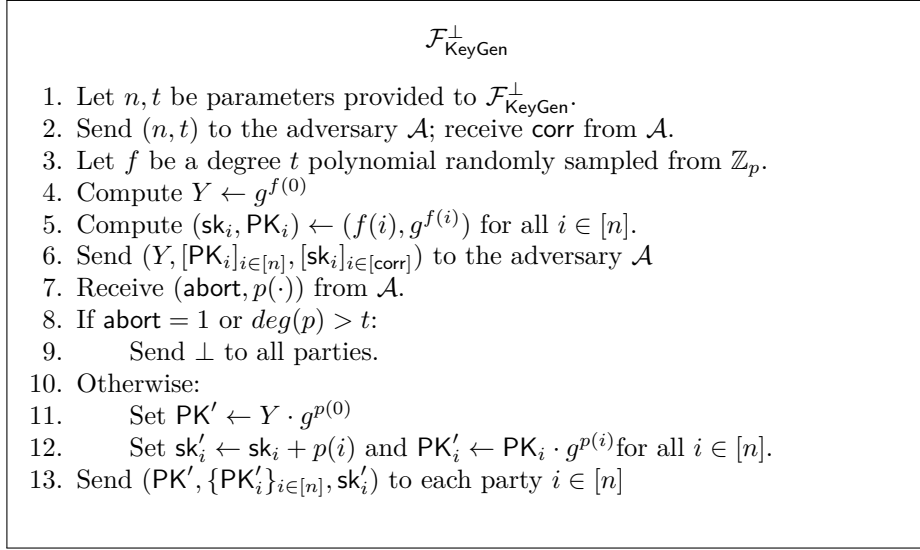


Fig. 1: Ideal functionality for key generation allowing adversarial abort, as defined by Kate et al. [KMSSSY26].

3. If a party collects a number of votes and deduces that there is no way to meet the necessary threshold for any of the collected votes, then it immediately aborts instead of waiting for timeouts to abort the process.

2.3 Distributed Key Generation

We follow the definition and idealized notion of security $\mathcal{F}_{\text{KeyGen}}^\perp$ for a distributed key generation (DKG) protocol where the adversary is allowed to bias the output key with some (constrained) additive shift Δ , as presented by Katz [Katz23]. We give a stand-alone definition of security for $\mathcal{F}_{\text{KeyGen}}^\perp$. Because our proof for PedPoP+ uses only online straight-line simulation, our proof with respect to $\mathcal{F}_{\text{KeyGen}}^\perp$ in the standalone setting is also by extension secure in the Universal Composability (UC) framework [KushilevitzLR10]. We assume an adversary that performs only static corruptions.

2.4 Online-Extractable & Straight-Line Simulation Proof System

Proving the security of our protocol requires a zero-knowledge proof system that must be online-extractable as in the UC framework, rewinding is not allowed.

One possible direction is to use zero-knowledge proofs described in [GOLDEN]. The zero-knowledge proof relation $\mathcal{R}_P$, described in the following relation demonstrates that an ElGamal ciphertext is correct. The prover shows either that it knows the plaintext sk of an encryption, or that it knows the discrete logarithm

of a public value h chosen at random from $\mathbb{G}$ at the time of setup. The second proof demonstrates by inference that the prover knows sk , if the discrete logarithm problem is hard in $\mathbb{G}$.

$$\mathcal{R}_P = \{(g, h, \text{PK}, c = (C_0, C_1), z, R_g, R_h); (\text{sk}, r)\} \quad \text{iff}$$

$$(1) \quad C_0 = g^r; \quad C_1 = h^r \cdot g^{\text{sk}}$$

$$(2) \quad g^z = R_g \cdot (g^{\text{sk}})^{c'}$$

$$(3) \quad h^z = R_h \cdot (h^{\text{sk}})^{c'}$$

SR: Ideal functionality can be found in [GOLDEN]

3 PedPoP

In the rest of this document, we denote by k the identifier of the participant running the protocol, by t the signing threshold of the participants, and by n the total number of participants. We assume, w.l.o.g. that the participants' identifiers range from 1 to n and that no two participants have the same identifier. We also assume that the parameters provided to the scheme are coherent, that is, they validate some intuitive set of rules such as $n > 2$, $k \in [n]$ and $t \leq n$.

We depict PedPoP in Figure 3 and refer the reader to [EPRINT:KomGol20] for proof details. In a nutshell, the protocol runs in two rounds. In the first round, the secret coefficients are picked at random and committed to then broadcast along with a Schnorr signature acting as a proof of knowledge of the secret key share associated to the verification key share. In the second round, the polynomial created using the secret coefficients is evaluated on the participant identifiers. Each evaluation is shared privately with the corresponding peer participant. Finally, after receiving the evaluations, a sequence of checks is performed to ensure the validity of the signatures, commitments, and polynomial evaluations received. The master verification key, verification key shares, and private verification key are all computed and returned by the protocol.

4 PedPoP+

Chelsea: TODO give domains and co-domains of hash functions, and update proof to match

PedPoP+ is a modification of PedPoP in which three extra rounds of communications are added. This protocol could either be used as a DKG or as a Key Resharing scheme. We depict the key generation protocol in Figure 4 and key resharing in Figure 5.

We require an online-extractable non-interactive zero-knowledge proof (NIZK) $\hat{\text{NIZK}}$ -SchnorrR for the Schnorr relation SchnorrR, where

$$\text{SchnorrR} = \left\{ (g, \text{PK}) ; (\text{sk}) : \text{PK} = g^{\text{sk}} \right\}$$

Applying the non-interactive proof straight-line compiler given by Chen et al. [ChenDGMSS25] gives online-extractability and hence UC security.

$$\begin{aligned} \text{SchnorrR}_{\text{on}} = \left\{ (g, h, \text{PK}, R_1, R_2, z) ; (\text{sk}, r) \right\} \\ \text{iff} \quad \begin{aligned} (1) & g^z \stackrel{?}{=} \text{PK}^c \cdot R_1 \\ (2) & h^z \stackrel{?}{=} \text{PK}^c \cdot R_2 \\ (3) & c \stackrel{?}{=} \text{H}_R(g, h, \text{PK}, R_1, R_2) \end{aligned} \end{aligned}$$

where (g, h) are elements of the group $\mathbb{G}$ output by **Setup** such that the discrete logarithm relation is unknown.

4.1 Key Generation

In Round 0, a participant performs necessary checks to pick a random session identifier and broadcasts it. It then receives the broadcast session identifiers from its peers and hashes all identifiers into one single string in Round 1. The participant picks $t + 1$ random coefficients. Each coefficient a is committed to by computing g^a . The vector of commitments is hashed and sent to all the peers. Still in Round 1, the participant gives a proof of knowledge of αg^{sk} using the Schnorr signature scheme. In Round 2, the participant receives the hash values from its peers, stores them, and broadcasts the commitment vector along with the Schnorr signature. In Round 2, each participant evaluates the polynomial generated by the picked coefficients. The number of evaluations is equal to the number of participants currently contributing to the protocol. It then verifies the correctness of the signature and the well-formedness of the received hash value then privately shares each computed polynomial evaluation with the corresponding peer participant. Round 3 receives the evaluations and verifies their correctness, then computing the master verification key and the private signing key share. Finally, if all the checks pass, then a success string is broadcast. In Round 4, each participant collects n success strings, else the participant aborts.

4.2 Key Resharing

Key Resharing using PedPoP+ is similar to key generation, with the exception that participants distribute sub-shares of their secret share, as opposed to a random value, as is the case for key generation.

Allowing New Joiners It is possible to allow new participants to join the coalition of signers, in such a way that these participants also contribute to the total randomness of the shares, while keeping the original secret key the same.

We likewise show this approach in Figure 5.

In Round 1, we distinguish two cases: if the participant running the protocol is part of the new joining participants, then it simply sets the first coefficient (denoted with sk) to zero; otherwise, it linearizes its old share of secret key using Lagrange coefficients generated for common parties between old and new participants sets. More specifically, the function $\text{linearize}_k(\text{old_sk}, I)$ computes the product

$$\text{linearize}_k(\text{old_sk}, I) := \text{old_sk} \cdot \prod_{\substack{i \in I \\ i \neq k}} \frac{i}{i - k}$$

In Round 4, the computed verification key is checked to match the same old public key given as input to the protocol. Checking such invariant is due to the fact that honest new participants shall not affect the value of the old verification key as their initial private shares are set to zero.

5 Security

We next prove the security of PedPoP+ with respect to the notion of security defined by $\mathcal{F}_{\text{KeyGen}}^\perp$.

Theorem 1. *Assume NIZK-SchnorrR is a secure online-extractable zero-knowledge proof for the Schnorr relation SchnorrR. Then, for $t \leq n/2$, the PedPoP+ DKG is secure under aborts in the programmable random oracle model, and realizes the idealized notion $\mathcal{F}_{\text{KeyGen}}^\perp$.*

SR: Need to first define the DKG challenge $\dot{X}$ and to define τ .

Proof. To establish the proof of Theorem 1, we define a simulator SIM that is given black-box access to the static adversary $\mathcal{A}$.

Setup.

- SIM runs the adversary, and receives the set corr . SIM then sets $\text{hon} \leftarrow [n] \setminus \text{corr}$.
- SIM outputs corr to the functionality $\mathcal{F}_{\text{KeyGen}}^\perp$.
- SIM receives $\dot{X} \in \mathbb{G}$ from $\mathcal{F}_{\text{KeyGen}}^\perp$.
- SIM initializes the table $Q_2 \leftarrow \emptyset$

Simulating Random Oracle Queries. When queried by the adversary, SIM simulates its random oracles by lazy sampling.

- H_{sid}
- H_{com}
- H_{R}

Round 0. To simulate the initial round of the DKG, SIM follows the protocol honestly for each party $i \in \text{hon}$. **Chelsea:** Discuss how H_1 is simulated

Round 1. On input $\text{sid}_1, \dots, \text{sid}_n$, to simulate Round 1 of the DKG, SIM does the following:

- Derive sid honestly.
- For each honest party $i \in \text{hon}$, sample $H_i \leftarrow \mathbb{Z}_p$.
- When the adversary queries party $k \in \text{hon}$, output H_k .

Round 2. On input $(\{H_i\}_{i \in [n]})$, to simulate Round 2 of the DKG, SIM does the following:

- For each corrupt party $j \in \text{corr}$, look up $(\text{sid}, j, C_j[0]) \leftarrow Q_{\text{SchnorrR}}[H_j]$.
- If any H_j does not correspond to a valid entry or to the current sid , follow the remainder of Round 2 honestly for all honest participants. There is no need to simulate as honest parties will abort in a later round due to invalid validity checks.
- Otherwise, pick at random an identifier $\tau \leftarrow \text{hon}$, and follow Round 2 as defined in the protocol for all honest parties $i \in \text{hon}, i \neq \tau$,
- However, for party τ , simulate Round 2, as follows:
 1. Randomly sample corrupt parties' shares $\bar{x}_{\tau,j} \leftarrow \mathbb{Z}_p$ for all $j \in \text{corr}$.
 2. Set

$$C[0]_\tau \leftarrow \dot{X} \cdot \prod_{k \in \text{hon}, k \neq \tau} (C[0]_k)^{-1} \cdot \prod_{j \in \text{corr}} (C[0]_j)^{-1} \quad (1)$$

where $C[0]_k$ are from the honest parties $k \in \text{hon}, k \neq \tau$ polynomial commitments C .

3. Compute the Lagrange polynomials $\{L'_0(Z), \{L'_j(Z)\}_{j \in \text{corr}}\}$ for the set (of x -coordinates) $\{0 \cup \text{corr}\}$, as defined in Section 2.1.
4. For $\ell \in \{1, \dots, t-1\}$, set the remaining commitments, as follows:

$$C_\tau[\ell] \leftarrow C_\tau[0]^{L'_0(\ell)} \cdot \prod_{j \in \text{corr} \cup \{0\}} (g^{\bar{x}_{\tau,j}})^{L'_j(\ell)}$$

5. Generate a simulated zero-knowledge proof π_τ of SchnorrR for $C[0]_\tau$.
Chelsea: TODO describe how.
SR: i.e., by picking the Schnorr commitment/response pair (c, s) at random, computing $R \leftarrow g^s / \text{PK}_{\text{Schnorr}}^c$ then reprogramming the random oracle $H_R(R, \gamma_\tau) \leftarrow c$.
6. **SR:** Reprogram the random oracle $H_{\text{com}}(\text{sid}, k, C_k) \leftarrow H_k$
7. Output $\{(C_k, \pi_k)\}_{k \in \text{hon}}$.

Round 3. On input $\text{KeyGen}_3(\{C_i, \pi_i\}_{i \in [n]})$, to simulate Round 3 of the DKG, SIM does the following:

- Perform verification steps honestly; if any verification step fails, abort the protocol.
- **SR:**
- **Otherwise, for each corrupt party $j \in \text{corr}$ and $k \in \text{hon}$, look up $f_j(k) \leftarrow Q_{\text{SchnorrR}}[H_j]$.**
- **random sample honest $f_k(j)$ and send them/**
- **For party τ , compute $f_\tau(j) \leftarrow XXX - \sum_{k \in \text{hon}, k \neq \tau} \bar{x}_{\tau, j} - \sum_{j \in \text{corr}} f_j(k)$**

Round 4. On input $(\{f_i(k)\}_{i \in [n]})$, to simulate Round 4 of the DKG, SIM does the following:

- TODO

Round 5. On input $(\{s_i\}_{i \in [n]})$

□

- **Setup:** Accepts as input public parameters par , and a timeout constant δ .
 1. Initialize $Q_K^{(1)} \leftarrow \emptyset, Q_K^{(2)} \leftarrow \emptyset$.
 2. Generate public parameters $(\mathbb{F}_p, p, G, s, g) \leftarrow \text{GroupGen}(1^\lambda)$.
 3. Sample $h \leftarrow_{\$} G$.
 4. Set $\text{par} \leftarrow (\mathbb{F}_p, p, G, s, g, h)$.
- **KeyGen:** If $Q_K^{(1)}[\text{id}] \neq \perp$, return $\perp$. Else, sample $\text{sk} \leftarrow_{\$} \mathbb{Z}_s$, set $\text{PK} \leftarrow g^{\text{sk}}$. Set $Q_K^{(1)}[\text{id}] = \text{PK}$. Output (PK, sk) .
- **Register:** Accepts as input public key PK , identifier id' from party id , and proof π .
 1. Output $\perp$ if id' is not the identifier for party id .
 2. Output $\perp$ if $Q_K^{(2)}[\text{id}] \neq \perp$.
 3. Output $\perp$ if $\text{Verify}(\pi, \text{PK}) \neq 1$, where π proves the relation $\mathcal{R}_P$, as follows:

$$\mathcal{R}_P = \{(g, h, \text{PK}, c = (C_0, C_1), z, R_g, R_h); (\text{sk}, r)\} \quad \text{iff}$$

$$\begin{aligned} (1) \quad & C_0 = g^r; \quad C_1 = h^r \cdot g^{\text{sk}} \\ (2) \quad & g^z = R_g \cdot (g^{\text{sk}})^{c'} \\ (3) \quad & h^z = R_h \cdot (h^{\text{sk}})^{c'} \end{aligned}$$

4. Else, set $Q_K^{(2)}[\text{id}] = \text{PK}$.
- **Query:** Wait δ amount of time (where δ is the constant defined above), to give all parties enough time to complete registration. Then, output $Q_K^{(2)}$.

Fig. 2: Ideal functionality for key generation allowing adversarial abort, as defined by Kate et al. [KMSSSY26].

$\text{KeyGen}_1(k, t, n)$	$\text{KeyGen}_3(\{f_i(k), (C_i, \pi_i)\}_{i \in [n]})$
$(\alpha, a_1, \dots, a_t) \leftarrow \mathbb{Z}_p^t$ <i>// local party has index k</i> $A_0 \leftarrow g^\alpha$ for $i \in [t]$ do $A_i \leftarrow g^{a_i}$ $C_k \leftarrow (A_0, \dots, A_t)$ for $j \in [n]$ do $f(j) \leftarrow \alpha + \sum_{i=1}^t a_i j^i$ $\pi_k \leftarrow \text{schnorr}[\text{H}_1].\text{Sign}(\alpha, \Phi)$ sendbroadcast $((C_k, \pi_k))$	for $i \in [n], i \neq k$ do <i>// Check proof of knowledge of sk</i> require $\text{schnorr}[\text{H}_1].\text{Verify}(C_i[0], \Phi, \pi_i) \neq 1$ <i>// Check correctness of share</i> require $g^{f_i(k)} \neq \prod_{j=0}^t C_i[j]^{k^j}$ for $i \in [n]$ do <i>// Derive each participant's public key share</i> $\text{PK}_i \leftarrow \prod_{j=1}^n \prod_{\ell=0}^{t-1} C_j[\ell]^{i^\ell}$ $\text{sk}_k \leftarrow \sum_{i=1}^n f_i(k)$ $\text{PK} \leftarrow \prod_{i=1}^n C_i[0]$ return $(\text{PK}, \{\text{PK}_i\}_{i=1}^n, \text{sk}_k)$
KeyGen_2	
for $i \in [n]$ do $\text{send}(i, f(i))$	

Fig. 3: The two-round PedPop DKG. The public parameters par are given as input to all algorithms and protocols. schnorr is the single-party Schnorr signature scheme. Φ is a context string that aims to prevent replay attacks. It contains the identifier of the signer, the verification key share

KeyGen₀ (k, t, n) require $n > 2$ require $k \in [n]$ require $t \leq n$ $\text{sid}_i \leftarrow_{\$} \{0, 1\}^\lambda$ sendbroadcast (sid_i) KeyGen₁ ($\text{sid}_1, \dots, \text{sid}_n$) $\text{sid} \leftarrow \text{H}_{\text{sid}}(\text{sid}_1, \dots, \text{sid}_n)$ $(\alpha, a_1, \dots, a_t) \leftarrow_{\$} \mathbb{Z}_p^t$ $A_0 \leftarrow g^\alpha$ for $i \in [t]$ do $A_i \leftarrow g^{a_i}$ $C_k \leftarrow (A_0, \dots, A_t)$ $H_k \leftarrow \text{H}_{\text{com}}(k, C_k, \text{sid})$ $\gamma_k \leftarrow (\text{sid}, k, C_k[0])$ $\pi_k \leftarrow_{\$} \text{NI\hat{Z}K-Straight}[\text{H}_R].\text{Prove}(\alpha, \gamma_k)$ for $i \in [n]$ send (i, H_k) KeyGen₂ ($\{H_i\}_{i \in [n]}$) sendbroadcast (C_k, π_k)	KeyGen₃ ($\{C_i, \pi_i\}_{i \in [n]}$) for $j \in [n]$ $f_k(j) \leftarrow \alpha + \sum_{i=1}^t a_i j^i$ for $i \in [n], i \neq k$ $\gamma_i \leftarrow (\text{sid}, k, C_i[0])$ require $\text{NI\hat{Z}K-Straight}[\text{H}_R].\text{Verify}(C_i[0], \gamma_i, \pi_i)$ require $\text{H}_2(\text{sid}, i, C_i) = H_i$ send ($i, f_k(i)$) KeyGen₄ ($\{f_i(k)\}_{i \in [n]}$) for $i \in [n], i \neq k$ require $g^{f_i(k)} = \prod_{j=0}^t C_i[j]^{k^j}$ (No derivation for participants public key) $\text{sk}_k \leftarrow \sum_{i=1}^n f_i(k)$ $\text{PK} \leftarrow \prod_{i=1}^n C_i[0]$ sendbroadcast (“success”, sid) KeyGen₅ ($\{(\text{“success”}_i, \text{sid}_i)\}_{i \in [n]}$) for $i \in [n], i \neq k$ require ($\text{“success”}_i, \text{sid}_i$) = ($\text{“success”}, \text{sid}$) return (PK, sk_k)
--	---

Fig. 4: The five-round DKG PedPoP+. See Section 2.2 for definitions of **send** and **sendbroadcast**.

$\text{KeyGen}_0(k, t, n, \boxed{old_sk_k, old_pk, old_signers, old_t})$ require $n > 2$ require $k \in [n]$ require $t \leq n$ $I \leftarrow [n] \cap old_signers$ require $old_t \leq I $ $sid_i \leftarrow \$ \{0, 1\}^\lambda$ sendbroadcast (sid_i) $\text{KeyGen}_1(sid_1, \dots, sid_n)$ $sid \leftarrow H_1(sid_1, \dots, sid_n)$ $(a_1, \dots, a_t) \leftarrow \$ \mathbb{Z}_p^{t-1}$ if $k \notin old_signers$ $\alpha \leftarrow 0$ else $\alpha \leftarrow \text{linearize}_k(old_sk_k, I)$ $A_0 \leftarrow g^\alpha$ for $i \in [t]$ $A_i \leftarrow g^{a_i}$ $C_k \leftarrow (A_0, \dots, A_t)$ $H_k \leftarrow H_2(k, C_k, sid)$ $\gamma_k \leftarrow (sid, k, C_k[0])$ $\pi_k \leftarrow \$ \text{schnorr}[H_3].\text{Sign}(\alpha, \gamma_k)$ for $i \in [n]$ $\text{send}(i, H_k)$ $\text{KeyGen}_2(\{H_i\}_{i \in [n]})$ sendbroadcast (C_k, π_k)	$\text{KeyGen}_3(\{C_i, \pi_i\}_{i \in [n]})$ for $j \in [n]$ $f_k(j) \leftarrow \alpha + \sum_{i=1}^t a_i j^i$ for $i \in [n], i \neq k$ $\gamma_i \leftarrow (sid, k, C_i[0])$ require $\text{schnorr}[H_3].\text{Verify}(C_i[0], \gamma_i, \pi_i)$ require $H_2(sid, i, C_i) = H_i$ send ($i, f_k(i)$) $\text{KeyGen}_4(\{f_i(k)\}_{i \in [n]})$ for $i \in [n], i \neq k$ require $g^{f_i(k)} = \prod_{j=0}^t C_i[j]^{k^j}$ $sk_k \leftarrow \sum_{i=1}^n f_i(k)$ $PK \leftarrow \prod_{i=1}^n C_i[0]$ $\boxed{\text{require } PK = old_pk}$ sendbroadcast ("success", sid) $\text{KeyGen}_5(\{("success_i", sid_i)\}_{i \in [n]})$ for $i \in [n], i \neq k$ require ("success", sid_i) = ("success", sid) return (PK, sk_k)
--	--

Fig. 5: The five-round Resharing Scheme Based on PedPoP+. Differences with the DKG protocol PedPoP+ are highlighted in a box. See Section 2.2 for definitions of **send** and **sendbroadcast**.